

SIMATS ENGINEERING
CO4 - ASSESSMENT TOOL 2
Industry Problem Solving Task

Name of the Student	Guggilla Venkatasai
Reg. No	192425388
GitHub Link	https://github.com/venkatasai-eng/MLA0403-Deep-learning

COURSE NAME: Deep Learning **COURSE CODE:** MLA0403 **SLOT:** A

Course Outcome Covered

CO 4: Learners will be able to interpret and apply probabilistic graphical models — Bayesian Networks, Markov Random Fields, Hidden Markov Models and Conditional Random Fields — to represent dependencies among random variables and to carry out inference and learning within such models. The outcome further covers using these formalisms for decision-making under uncertainty in practical settings. (BL-6)

Course Outcome Weightage: 10% **Duration:** 90 minutes

Assessment Tool Weightage: 40% **Marks:** 25

Code of Conduct

I Guggilla Venkatasai(192425388) declare that the work presented in this submission is entirely my own and complies with the guidelines set for this assessment.

I acknowledge that any breach of academic integrity will attract disciplinary consequences.

Name of the student: Guggilla Venkatasai **Signature of the student:**



Rubric Summary

Criteria	Understanding of Industry Problem (2)	Application of Engineering Concepts (2.5) Solution Design (2.5)	Use of Tools (2)	Analysis (1)	Total (10)
Weightage	20%	25% 25%	20%	10%	100%
Q1					
Q2					
Q3					
Q4					
Q5					

Total

Questions: 05

Total Marks: 25

Q1. Transfer Learning – Skin Cancer Detection

A healthcare startup wants to build an AI-based screening tool that classifies dermoscopic images to support early skin-cancer detection. Because curating a large, expertly labelled medical-imaging dataset is expensive and slow, transfer learning from a network already trained on natural images is a practical way to reach usable accuracy sooner.

1. Understanding of the Industry Problem

The core task is binary classification of dermoscopic images into benign and malignant categories. Two constraints dominate the problem: labelled dermatology data is scarce and expensive to acquire (each image needs expert dermatologist annotation), and training a deep CNN from random initialisation on a small dataset tends to overfit rather than generalise — the model memorises training examples instead of learning transferable lesion patterns.

There is also a strong class-imbalance dimension in real dermatology datasets: malignant cases are typically far rarer than benign ones, so the model must be evaluated (and often trained) with this imbalance explicitly in mind rather than assuming a balanced split. This makes the case for reusing features learned on a large, unrelated image corpus such as ImageNet, where millions of labelled natural images have already taught a network general visual patterns.

2. Proposed Solution

ResNet50, pre-trained on ImageNet, is a strong candidate backbone because its residual (skip) connections let very deep networks train without the gradients vanishing, and the network already encodes general-purpose visual patterns — edges, textures and shapes — that transfer well to lesion images. Compared with a shallower CNN, ResNet50's depth gives it a richer hierarchy of features: early layers detect edges and colour gradients, middle layers detect textures, and deeper layers detect complex shapes — exactly the kind of structure needed to distinguish irregular lesion borders and colour variation, which are key malignancy indicators.



Figure 1.1 — End-to-end pipeline: image capture through to the malignant/benign prediction

3. Frozen and Fine-Tuned Layers

Rather than training every parameter, the model is adapted in stages so that generic low-level features are preserved while higher-level features are adjusted to lesion-specific patterns:

- Early convolutional layers stay frozen — they already capture edges and textures common to all images, and retraining them on a small dataset would only risk destroying useful, well-generalised features.
- Middle layers begin frozen and are candidates for later unfreezing, once the new head has learned reasonable weights and stopped producing large, disruptive gradients.
- The final convolutional block is fine-tuned so the network can specialise on lesion characteristics such as border irregularity, colour asymmetry and texture.
- The original 1000-class ImageNet classification head is discarded, since the target task (2-class benign/malignant) has nothing to do with ImageNet's object categories.
- A new dense layer with a sigmoid activation is added and trained from scratch for the benign/malignant decision, since sigmoid naturally outputs a probability for binary classification.



Figure 1.2 — Which parts of ResNet50 stay frozen versus which are retrained

Once the new head has stabilised, a subset of the deeper ResNet50 layers can be unfrozen and trained further with a small learning rate (typically 10–100× smaller than the initial rate), allowing the network to gently specialise without destroying the pre-trained weights. This staged approach — train the head first, then fine-tune gradually — is a standard and effective transfer-learning practice.

4. Tools and Technologies

- Python, TensorFlow / Keras
- ResNet50 (pre-trained on ImageNet)
- OpenCV and NumPy for image handling
- A dermoscopic dataset such as ISIC (International Skin Imaging Collaboration)
- Augmentation — rotation, flipping, cropping, brightness jitter — to enlarge the effective training set and reduce overfitting further
- Class-weighting or oversampling techniques (e.g. SMOTE) to counter the benign/malignant class imbalance

5. Analysis and Performance

Because most of the network's weights are reused rather than learned from zero, training converges faster and the smaller number of trainable parameters reduces the risk of overfitting on a limited dataset.

Suitable evaluation metrics include accuracy, precision, recall/sensitivity, F1-score, ROC-AUC and the confusion matrix. In a clinical screening context, recall/sensitivity deserves particular weight, since failing to flag a

malignant case (a false negative) is far costlier than a false alarm (a false positive) — a missed malignant lesion could delay life-saving treatment, whereas a false alarm simply triggers a follow-up examination. For this reason, the decision threshold on the sigmoid output is often tuned below the default 0.5 to favour recall over raw accuracy.

Q2. DenseNet and PixelNet – Road Scene Analysis

A smart-city initiative needs a road-scene understanding system that labels every pixel of a street image as road, pedestrian, vehicle, traffic sign or background — a semantic segmentation task rather than simple object detection.

1. Understanding of the Industry Problem

For autonomous driving and traffic-monitoring use cases, knowing that a vehicle is present somewhere in an image is not enough — the system needs pixel-precise boundaries to support navigation, lane-keeping and safety decisions. This raises the bar from bounding-box detection to dense, per-pixel classification. A bounding box around a pedestrian, for example, includes background pixels inside the box, which is insufficient when the vehicle needs to know exactly where the drivable road surface ends and the pedestrian begins.

The problem is also multi-scale: a nearby vehicle occupies a large area of the frame while a distant traffic sign may be only a few pixels wide, so the architecture must preserve fine detail alongside broader contextual understanding.

2. Proposed Architecture

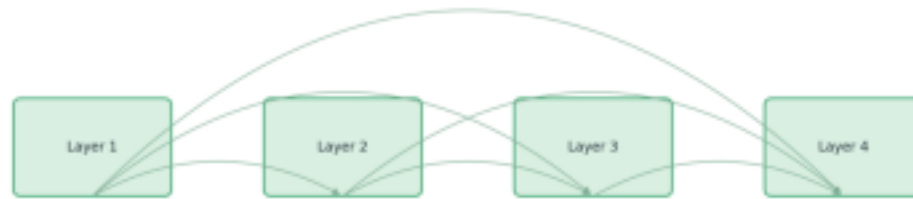
A two-stage design pairs DenseNet as the feature-extraction backbone with a PixelNet-style head for pixel-wise prediction.



Figure: 2.1 — Road image to segmentation map via DenseNet feature extraction and PixelNet pixel classification

3. Role of DenseNet

Conventional CNNs pass information strictly from one layer to the next. DenseNet instead connects every layer to all layers that follow it, so features computed early in the network are directly available deep inside it.



Each layer receives feature maps from all preceding layers

Figure 2.2 — Dense connectivity: later layers reuse feature maps from every earlier layer

- Stronger feature reuse across the network
- Improved gradient flow during backpropagation, since each layer has a direct path back to earlier layers
- Less redundant re-learning of the same low-level features
- Solid representation of edges, textures and object boundaries — useful for vehicles, pedestrians, lane markings and signage

4. Role of PixelNet

PixelNet takes the features DenseNet extracts and assigns each individual pixel to a

class:

- Road

- Pedestrian

- Vehicle

- Traffic sign

- Background

Combining every pixel's predicted label produces the final segmentation map, where distinct regions correspond to distinct object categories. Upsampling restores the spatial resolution lost during feature extraction so the output map matches the original image dimensions pixel-for-pixel.

5. Computational Efficiency

Because dense connections let the network reuse features instead of re-deriving them at every stage, parameter usage tends to be more efficient than in a comparably deep plain CNN. Pixel-level prediction, meanwhile, preserves the fine detail needed to segment small objects such as traffic signs and distant pedestrians. This efficiency matters for smart-city deployment, where inference may need to run on multiple camera feeds concurrently or on edge hardware with limited compute.

6. Tools

- Python, TensorFlow/Keras or PyTorch
- DenseNet backbone
- PixelNet-style segmentation head

- OpenCV
- Cityscapes or a comparable road-scene dataset
- GPU/CUDA for training

7. Analysis and Performance

- Pixel accuracy
- Mean Intersection over Union (mIoU)
- Precision and recall
- F1-score
- Inference time

mIoU is especially informative here because it directly measures how well the predicted region for each class overlaps with the ground-truth region, rather than just counting correctly labelled pixels overall — a metric like raw pixel accuracy can look artificially high simply because 'road' and 'background' dominate most frames, while mIoU still penalises poor segmentation of rarer classes like traffic signs. Inference time is equally critical for real time traffic monitoring, since a segmentation model that is accurate but too slow cannot support live navigation decisions.

Q3. Bidirectional RNN and Encoder–Decoder – Multilingual Chatbot

A global customer-support team wants a chatbot that can translate customer queries across languages while keeping the original meaning intact. A sequence-to-sequence encoder–decoder built around bidirectional RNNs is well suited to this.

1. Understanding of the Industry Problem

Customer messages can be long and context-dependent, so the system needs to read and understand a full sentence before it starts producing a translation — for example, correctly interpreting "Where can I change my booking?" requires the relationship between all of its words, not just individual word meanings. Customer-support text also carries domain-specific vocabulary (booking references, product names, error codes) and can include incomplete or informal phrasing, which raises the bar for what 'understanding the sentence' actually requires.

2. Proposed Architecture

The input sentence is embedded and passed through a bidirectional RNN encoder that reads it in both directions simultaneously; the resulting representation then drives a decoder RNN that generates the translated output word by word.

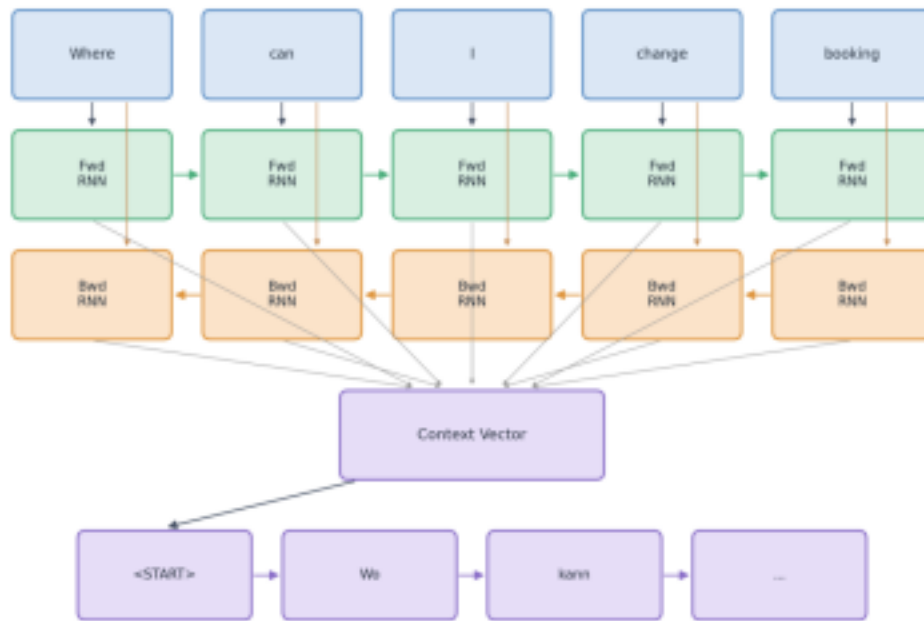


Figure 3.1 — Forward and backward encoder passes combine into a context vector consumed by the decoder **3.**

Encoder

The encoder processes the sentence left-to-right and right-to-left at the same time. For a sentence like "The customer cancelled the order yesterday," a word's meaning can depend on words that come both before and after it, so combining both passes gives a richer representation than a single-direction RNN could. The forward pass captures how meaning builds up to a point; the backward pass captures how later words retroactively clarify earlier ones — together they give each word a context window spanning the whole sentence.

4. Decoder

The decoder unrolls the context vector into the output sentence one token at a time. Adding an attention mechanism lets the decoder weight the most relevant parts of the encoded input differently at each generation step, rather than relying on a single fixed-size summary. This is particularly valuable for longer customer messages, where compressing an entire multi-clause sentence into one fixed vector would otherwise lose detail — attention lets the decoder 'look back' at the specific source words most relevant to the word it is currently generating.

5. Advantages of Bidirectional Processing

- Captures both past and future context around each word
- Improves understanding of sentence structure
- Handles context-dependent word meanings more reliably
- Scales better to longer sentences
- Produces a richer encoder representation for the decoder to draw on

6. Tools

- Python, TensorFlow/Keras or PyTorch
- NLP preprocessing and tokenisation libraries
- Pre-trained or learned word embeddings
- BLEU score for translation quality
- Parallel multilingual datasets

7. Analysis

- BLEU score
- Translation accuracy
- Perplexity
- Validation loss
- Human evaluation

Benchmarking against a plain unidirectional RNN is a useful sanity check: a higher BLEU score and lower validation loss for the bidirectional model would confirm the extra context is actually helping. Human evaluation remains important alongside BLEU, since automatic metrics can miss fluency or tone issues that matter in a customer-facing support context.

Q4. BPTT and LSTM – Stock Price Prediction

A financial analytics firm wants to forecast near-term stock prices from historical market data. Because prices form a time-ordered sequence with dependencies that can stretch back many steps, an LSTM-based recurrent network is a natural fit.

1. Understanding of the Industry Problem

Price movements often depend on patterns established well before the current time step — for instance, a stock's volatility over the past month, or a slow multi-week trend, can influence tomorrow's price as much as yesterday's close. A plain RNN struggles here because of the vanishing-gradient problem, which makes it hard for the network to retain information from many steps in the past; gradients shrink exponentially as they are propagated backward through a long sequence, so early time steps effectively stop influencing the weight updates.

2. Proposed LSTM Architecture

Historical data — opening, high, low and closing prices plus trading volume — is preprocessed into sequences of past days, which an LSTM layer consumes to predict the next day's price via a final dense layer.



Figure 4.1 — LSTM cell: forget, input and output gates regulate what enters and leaves the cell state

3. LSTM Structure

- Forget gate — decides which information in the cell state is no longer relevant and should be discarded
- Input gate — decides which new information from the current step should be added to the cell state
- Cell state — the running memory that carries information across time steps
- Output gate — decides what part of the cell state to expose as the current output

4. Backpropagation Through Time

Training an LSTM (or any recurrent network) relies on backpropagation through time: the network is unrolled across the input sequence, a prediction error is computed at the end, and that error is propagated backward through every preceding time step so the optimiser can update the weights.



Figure 4.2 — BPTT: the prediction error at Day 4 is propagated back through Days 3, 2 and 1

5. Why LSTM Instead of a Traditional RNN?

- A plain RNN struggles with long-term dependencies; an LSTM's gates are explicitly designed to preserve them.
- Plain RNNs are more exposed to vanishing gradients; LSTM gating controls how gradients and information flow.
- A plain RNN's memory mechanism is simple; the LSTM's cell state gives it persistent, selectively-updated memory.
- This makes LSTM the better default for long, noisy financial sequences.

6. Tools

- Python, TensorFlow/Keras
- Pandas, NumPy for data handling
- Matplotlib for visualisation
- Historical stock market datasets
- GPU acceleration for faster training

7. Performance Analysis

- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)

Plotting actual versus predicted prices alongside these metrics gives an intuitive check on model quality; lower RMSE/MAE values indicate a tighter fit. RMSE is particularly useful because it penalises large errors more heavily than MAE, which matters in finance since a single large mis-prediction can be more costly than several small ones.

Conclusion

The gated structure of an LSTM lets it retain useful signal over longer horizons than a plain RNN can manage, and BPTT is the mechanism that trains it to do so by propagating error across the full sequence.

Q5. Integrated Deep Learning Solution – Automatic Video Captioning

A streaming platform wants to auto-generate captions for uploaded video content. This calls for an architecture that combines transfer learning for visual understanding with a sequence model for language generation — a CNN feature extractor feeding an LSTM encoder–decoder.

1. Understanding of the Industry Problem

A video carries far more information than a single image, so the system first needs to interpret what is happening across frames and then translate that understanding into a coherent natural-language sentence — for instance, turning footage of someone playing football into the caption "A person is playing football." This requires combining spatial understanding (what objects and actions appear in each frame) with temporal understanding (how those frames relate to one another over time) and finally language generation (turning that combined understanding into fluent text).

2. Proposed Architecture



Figure 5.1 —
Frames are converted to visual features, summarised by an LSTM encoder, and turned into text by an LSTM decoder

3. Transfer Learning Component

A pre-trained CNN such as ResNet50 or EfficientNet, already exposed to a broad range of visual concepts, extracts a feature vector from each sampled frame:

- Extract frames from the video at regular intervals
- Resize and normalise each frame
- Pass each frame through the pre-trained CNN
- Collect the resulting feature vectors
- Feed the feature sequence into the LSTM encoder

The CNN's original classification head is dropped since the goal here is a feature representation, not an image class label.

4. LSTM Encoder

The encoder receives the sequence of per-frame feature vectors and learns how they relate to one another over time, producing a single contextual summary of the video's content — for example, recognising that a sequence of frames showing a leg swinging toward a ball represents the action of kicking, rather than treating each frame as an isolated image.

5. LSTM Decoder

The decoder converts that summary into a sequence of words, generating the caption token by token from a start marker to an end marker, learning during training to associate visual patterns with the corresponding caption text. An attention mechanism can let the decoder focus on the frames most relevant to the word currently being generated — for instance, attending most strongly to the frame showing contact with the ball when generating the word "kicking."

6. Role of Each Component

- Video frames — supply the raw visual information
- Pre-trained CNN — extracts per-frame visual features
- Transfer learning — cuts down the training effort needed for visual understanding
- LSTM encoder — models how the video's content evolves over time
- LSTM decoder — turns the visual summary into words
- Attention — links specific frames to specific words

- Tokenizer — converts words to and from numerical form for the model

7. Tools

- Python, TensorFlow/Keras or PyTorch
- OpenCV for frame handling
- Pre-trained ResNet50 or EfficientNet
- LSTM layers
- An NLP tokenizer
- GPU/CUDA
- Video-caption datasets such as MSR-VTT or MSVD

8. Performance Analysis

- BLEU
- ROUGE
- METEOR
- CIDEr
- Human evaluation

Testing across varied video categories, lighting conditions, camera motion and caption lengths helps confirm the system generalises rather than overfitting to a narrow style of footage. CIDEr is particularly relevant for captioning tasks since it was designed specifically to correlate with human consensus judgement of caption quality, unlike BLEU which was originally built for machine translation.

9. Advantages of the Integrated Solution

- Strong visual feature extraction via transfer learning
- Reduced training time compared to learning visual features from scratch
- Temporal understanding of the video contributed by the LSTM encoder
- Fluent natural-language caption generation from the decoder
- Better handling of sequential, multi-frame video information

Conclusion

Transfer learning supplies strong visual representations without the cost of training a CNN from the ground up, the LSTM encoder captures how the video unfolds over time, and the LSTM decoder turns that understanding into readable captions. Attention sharpens the link between individual frames and the words generated for them, making this integrated architecture a practical fit for automatic video captioning.

Rubrics for Calculation

Criteria	Weight	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	--------	---------------------	----------------------	----------------------	---------------------

Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	20%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis